

DocProcessor

Belgelendirmeyi QA otomasyonu için doğrulanabilir bir özellik haritasına dönüştürün.

Özet

DocProcessor, proje belgelerini yükleyen, yapılandırılmış özellik haritaları oluşturan ve doğrulama kapsamını izleyen, bağımsız ve tamamen ayrıştırılmış bir Go modülüdür. LLM ajanlarıyla akıllı özellik çıkarma için tasarlanmış olsa da çevrimdışı kullanım için sezgisel çıkarma özelliğini de içerir.

Kısa açıklama

Belge işleme ve özellik haritası çıkarma için projeye bağımlı olmayan bir Go modülü. Belgeleri yapılandırılmış özellik haritalarına dönüştürür ve hangi özelliklerin doğrulandığını izler — akıllı çıkarma için LLM ajanlarını kullanır veya çevrimdışı durumlarda sezgisel yöntemlerle çalışır — QA otomasyonuna, “gerçekle her zaman uyumlu” garantisi sunar.

Uzun açıklama

Her yazılım ekibi aynı yavaş yalanla yaşamak zorunda kalır: belgeler özellikler vaat eder, testler bunlara yakın bir şeyi kapsar ve kimse bu ikisinin aynı ürünü tarif edip etmediğini güvenle söyleyemez. DocProcessor, bu boşluğu görünür ve ölçülebilir kılmak için var. Bir projenin belgeleri verildiğinde, ürünün yapmayı vaat ettiği her şeyin numaralandırılmış, makine tarafından okunabilir bir modeli olan yapılandırılmış bir özellik haritası oluşturur ve doğrulama kapsamını bu haritaya karşı izler. Böylece “bu belgelenmiş özellik gerçekten kanıtlandı mı?” sorusu, koridor tartışmalarından çıkıp bir yanıtı olan bir sorguya dönüşür. Kasıtlı olarak çift modludur: LLM ajanları mevcut olduğunda akıllı, anlamsal özellik çıkarma için kullanılır; çevrimdışı kullanım için ise sezgisel bir ayrıştırıcıya geri döner. Bu sayede, bir modelin

varlığına kesin bağımlılık olmaz ve hava boşluklu bir CI işinde ya da bir geliştiricinin uçak modundaki dizüstü bilgisayarında aynı şekilde çalışır.

Mimari olarak, bağımsız, proje farkındalığı olmayan ve tamamen ayrıştırılmış bir Go modülüdür (CONST-051(B)): Hiçbir projeye özgü değer içermez ve tüketiciler tarafından eşit kod tabanlı bir alt modül olarak entegre edilir. Böylece herhangi bir proje, başkalarının varsayımlarını devralmadan benimseyebilir. Ayrıca, başkalarına uyguladığı standardı kendisine de uygular — kendi iddiaları, “anti-blöf” sözleşmesi (CONST-035) ve tam otomasyon kapsamı kuralları (CONST-048) ile sınırlıdır. Bu, README dosyasında duyurulan her yeteneğin, yalnızca sıfırla çıkmakla kalmayıp gerçek, son kullanıcı tarafından kullanılabilir davranışı doğrulayan otomatik bir test veya Challenge betiği tarafından sınandığı anlamına gelir. Kullanıcıya yönelik metinler, CONST-046 i18n çevirmen ara katmanından geçer. Tüm bunların amacı kapalı bir döngü oluşturmaktır: DocProcessor, HelixQA’un tamamladığı QA döngüsünün girdi tarafıdır — belgelerden özellik haritasını çıkarır, HelixQA ise her haritalanmış özelliği çalışma zamanı kanıtlarıyla doğrular. Böylece belgeler, testler ve yayınlanan davranış, sürümden sürüme sessizce ayrılmak yerine zorunlu olarak birleşir.

Neden geliştirdik

Belgeler ve testler birbirinden uzaklaşır: Belgeler hiçbir testin kanıtlamadığı özellikler vaat eder ve QA, “tamamlanmış”ın ne anlama geldiğini kolayca söyleyemez. DocProcessor, belgeleri makine tarafından okunabilir bir özellik haritasına dönüştürerek doğrulama kapsamının gerçekten vaat edilenlere karşı ölçülmesini sağlar.

İçerik

Neden bir oyun değiştirici?

Yazılım tesliminde en muğlak soruyu — “Yayınladığımız şey, söylediğimiz şeyle örtüşüyor mu?” — otomatikleştirilebilir ve sürekli denetlenebilir bir hale getiriyor. Üstelik bunu **AI** gibi katı bir bağımlılığa ihtiyaç duymadan yapıyor: Model varsa **LLM** tabanlı çıkarım, yoksa sezgisel yöntemler kullanıyor. Böylece çevrimdışı çalışan bir ortamdan tamamen otonom bir pipeline’a kadar her ortamda aynı güvence sağlanıyor.

Yenilikçi yönleri

- **Belgelendirme-özellik haritası çıkarımı** ve doğrulama kapsamı takibi.
- **Çift yönlü çıkarım:** LLM aracıyla ya da sezgisel/çevrimdışı yöntemlerle.
- **Projeye özel olmayan, sıfır konfigürasyonlu bağımsız çalışma** (CONST-051(B)).
- **Sahtekârlık önleyici öz doğrulama:** README iddiaları, testler/Meydan Okumalar ile destekleniyor (CONST-035/048).

Zorluklar ve çözümler

- **Model olmadan çalışabilme:** Çevrimdışı çalışmayı sağlayan sezgisel çıkarım yedek mekanizmasıyla çözüldü.
- **Belgelerin gerçeklikle uyumlu kalması:** Yapılandırılmış özellik haritaları ve doğrulama kapsamı takibi ile çözüldü; QA döngüsüne entegre edildi.
- **Yeniden kullanılabilirlik:** Katı bağımsızlık ve eşit kod tabanlı alt modül tüketimi ile sağlandı.
- **Kendi iddialarının güvenilirliği:** Her reklam edilen yetenek için sahtekârlık önleyici testler/Meydan Okumalar ile çözüldü.

Teknoloji yığını (neden ve nasıl)

- **Go (1.25+)** — modül çekirdeği; Apache-2.0 lisanslı.
- **LLM araçları** — akıllı anlamsal özellik çıkarımı (isteğe bağlı).
- **Sezgisel ayrıştırıcı** — çevrimdışı özellik çıkarımı yedeği.
- **Çok dilli Çevirmen (pkg/i18n)** — CONST-046 yerelleştirilmiş dizgiler.
- **Meydan Okuma altyapısı** — modülün kendi iddialarının sahtekârlık önleyici doğrulaması.